

15.1.17 CREATE PROCEDURE and CREATE FUNCTION Statements

```

CREATE
    [DEFINER = user]
    PROCEDURE [IF NOT EXISTS] sp_name ([proc_parameter[,...]])
    [characteristic ...] routine_body

CREATE
    [DEFINER = user]
    FUNCTION [IF NOT EXISTS] sp_name ([func_parameter[,...]])
    RETURNS type
    [characteristic ...] routine_body

proc_parameter:
    [ IN | OUT | INOUT ] param_name type

func_parameter:
    param_name type

type:
    Any valid MySQL data type

characteristic: {
    COMMENT 'string'
    | LANGUAGE SQL
    | [NOT] DETERMINISTIC
    | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
    | SQL SECURITY { DEFINER | INVOKER }
}

routine_body:
    Valid SQL routine statement

```

These statements are used to create a stored routine (a stored procedure or function). That is, the specified routine becomes known to the server. By default, a stored routine is associated with the default database. To associate the routine explicitly with a given database, specify the name as ***db_name.sp_name*** when you create it.

The CREATE FUNCTION statement is also used in MySQL to support loadable functions. See Section 15.7.4.1, “CREATE FUNCTION Statement for Loadable Functions”. A loadable function can be regarded as an external stored function. Stored functions share their namespace with loadable functions.

See Section 11.2.5, “Function Name Parsing and Resolution”, for the rules describing how the server interprets references to different kinds of functions.

To invoke a stored procedure, use the `CALL` statement (see Section 15.2.1, “CALL Statement”). To invoke a stored function, refer to it in an expression. The function returns a value during expression evaluation.

`CREATE PROCEDURE` and `CREATE FUNCTION` require the `CREATE ROUTINE` privilege. If the `DEFINER` clause is present, the privileges required depend on the `user` value, as discussed in Section 27.6, “Stored Object Access Control”. If binary logging is enabled, `CREATE FUNCTION` might require the `SUPER` privilege, as discussed in Section 27.7, “Stored Program Binary Logging”.

By default, MySQL automatically grants the `ALTER ROUTINE` and `EXECUTE` privileges to the routine creator. This behavior can be changed by disabling the `automatic_sp_privileges` system variable. See Section 27.2.2, “Stored Routines and MySQL Privileges”.

The `DEFINER` and `SQL SECURITY` clauses specify the security context to be used when checking access privileges at routine execution time, as described later in this section.

If the routine name is the same as the name of a built-in SQL function, a syntax error occurs unless you use a space between the name and the following parenthesis when defining the routine or invoking it later. For this reason, avoid using the names of existing SQL functions for your own stored routines.

The `IGNORE_SPACE` SQL mode applies to built-in functions, not to stored routines. It is always permissible to have spaces after a stored routine name, regardless of whether `IGNORE_SPACE` is enabled.

`IF NOT EXISTS` prevents an error from occurring if there already exists a routine with the same name. This option is supported with both `CREATE FUNCTION` and `CREATE PROCEDURE` beginning with MySQL 8.0.29.

If a built-in function with the same name already exists, attempting to create a stored function with `CREATE FUNCTION ... IF NOT EXISTS` succeeds with a warning indicating that it has the same name as a native function; this is no different than when performing the same `CREATE FUNCTION` statement without specifying `IF NOT EXISTS`.

If a loadable function with the same name already exists, attempting to create a stored function using `IF NOT EXISTS` succeeds with a warning. This is the same as without specifying `IF NOT EXISTS`.

See Function Name Resolution, for more information.

The parameter list enclosed within parentheses must always be present. If there are no parameters, an empty parameter list of `()` should be used. Parameter names are not case-sensitive.

Each parameter is an IN parameter by default. To specify otherwise for a parameter, use the keyword OUT or INOUT before the parameter name.

Note

Specifying a parameter as IN, OUT, or INOUT is valid only for a PROCEDURE. For a FUNCTION, parameters are always regarded as IN parameters.

An IN parameter passes a value into a procedure. The procedure might modify the value, but the modification is not visible to the caller when the procedure returns. An OUT parameter passes a value from the procedure back to the caller. Its initial value is NULL within the procedure, and its value is visible to the caller when the procedure returns. An INOUT parameter is initialized by the caller, can be modified by the procedure, and any change made by the procedure is visible to the caller when the procedure returns.

For each OUT or INOUT parameter, pass a user-defined variable in the CALL statement that invokes the procedure so that you can obtain its value when the procedure returns. If you are calling the procedure from within another stored procedure or function, you can also pass a routine parameter or local routine variable as an OUT or INOUT parameter. If you are calling the procedure from within a trigger, you can also pass NEW.*col_name* as an OUT or INOUT parameter.

For information about the effect of unhandled conditions on procedure parameters, see Section 15.6.7.8, “Condition Handling and OUT or INOUT Parameters”.

Routine parameters cannot be referenced in statements prepared within the routine; see Section 27.8, “Restrictions on Stored Programs”.

The following example shows a simple stored procedure that, given a country code, counts the number of cities for that country that appear in the city table of the world database. The country code is passed using an IN parameter, and the city count is returned using an OUT parameter:

```
mysql> delimiter //
```

```
mysql> CREATE PROCEDURE citycount (IN country CHAR(3), OUT cities INT)
```

```
    BEGIN
```

```
        SELECT COUNT(*) INTO cities FROM world.city
```

```
        WHERE CountryCode = country;
```

```
    END//
```

```
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> delimiter ;

mysql> CALL citycount('JPN', @cities); -- cities in Japan
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @cities;
+-----+
| @cities |
+-----+
|      248 |
+-----+
1 row in set (0.00 sec)

mysql> CALL citycount('FRA', @cities); -- cities in France
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @cities;
+-----+
| @cities |
+-----+
|       40 |
+-----+
1 row in set (0.00 sec)
```

The example uses the **mysql** client `delimiter` command to change the statement delimiter from `;` to `//` while the procedure is being defined. This enables the `;` delimiter used in the procedure body to be passed through to the server rather than being interpreted by **mysql** itself. See Section 27.1, “Defining Stored Programs”.

The `RETURNS` clause may be specified only for a `FUNCTION`, for which it is mandatory. It indicates the return type of the function, and the function body must contain a `RETURN value` statement. If the `RETURN` statement returns a value of a different type, the value is coerced to the proper type. For example, if a function specifies an `ENUM` or `SET` value in the `RETURNS` clause, but the `RETURN` statement returns an integer, the value returned from the function is the string for the corresponding `ENUM` member of set of `SET` members.

The following example function takes a parameter, performs an operation using an SQL function, and returns the result. In this case, it is unnecessary to use `delimiter` because the function definition contains no internal `;` statement delimiters:

```
mysql> CREATE FUNCTION hello (s CHAR(20))
-> RETURNS CHAR(50) DETERMINISTIC
-> RETURN CONCAT('Hello, ',s,'!');
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT hello('world');
+-----+
| hello('world') |
+-----+
| Hello, world!  |
+-----+
1 row in set (0.00 sec)
```

Parameter types and function return types can be declared to use any valid data type. The `COLLATE` attribute can be used if preceded by a `CHARACTER SET` specification.

The ***routine_body*** consists of a valid SQL routine statement. This can be a simple statement such as `SELECT` or `INSERT`, or a compound statement written using `BEGIN` and `END`. Compound statements can contain declarations, loops, and other control structure statements. The syntax for these statements is described in Section 15.6, “Compound Statement Syntax”. In practice, stored functions tend to use compound statements, unless the body consists of a single `RETURN` statement.

MySQL permits routines to contain DDL statements, such as `CREATE` and `DROP`. MySQL also permits stored procedures (but not stored functions) to contain SQL transaction statements such as `COMMIT`. Stored functions may not contain statements that perform explicit or implicit commit or rollback. Support for these statements is not required by the SQL standard, which states that each DBMS vendor may decide whether to permit them.

Statements that return a result set can be used within a stored procedure but not within a stored function. This prohibition includes `SELECT` statements that do not have an `INTO var_list` clause and other statements such as `SHOW`, `EXPLAIN`, and `CHECK TABLE`. For statements that can be determined at function definition time to return a result set, a Not allowed to return a result set from a function error occurs (`ER_SP_NO_RETSER`). For statements that can be determined only at runtime to return a result set, a PROCEDURE %s can't return a result set in the given context error occurs (`ER_SP_BADSELECT`).

`USE` statements within stored routines are not permitted. When a routine is invoked, an implicit `USE db_name` is performed (and undone when the routine terminates). This causes the routine to have the given default database while it executes. References to objects in databases other than the routine default database should be qualified with the appropriate database name.

For additional information about statements that are not permitted in stored routines, see Section 27.8, “Restrictions on Stored Programs”.

For information about invoking stored procedures from within programs written in a language that has a MySQL interface, see Section 15.2.1, “CALL Statement”.

MySQL stores the `sql_mode` system variable setting in effect when a routine is created or altered, and always executes the routine with this setting in force, *regardless of the current server SQL mode when the routine begins executing*.

The switch from the SQL mode of the invoker to that of the routine occurs after evaluation of arguments and assignment of the resulting values to routine parameters. If you define a routine in strict SQL mode but invoke it in nonstrict mode, assignment of arguments to routine parameters does not take place in strict mode. If you require that expressions passed to a routine be assigned in strict SQL mode, you should invoke the routine with strict mode in effect.

The `COMMENT` characteristic is a MySQL extension, and may be used to describe the stored routine. This information is displayed by the `SHOW CREATE PROCEDURE` and `SHOW CREATE FUNCTION` statements.

The `LANGUAGE` characteristic indicates the language in which the routine is written. The server ignores this characteristic; only SQL routines are supported.

A routine is considered “deterministic” if it always produces the same result for the same input parameters, and “not deterministic” otherwise. If neither `DETERMINISTIC` nor `NOT DETERMINISTIC` is given in the routine definition, the default is `NOT DETERMINISTIC`. To declare that a function is deterministic, you must specify `DETERMINISTIC` explicitly.

Assessment of the nature of a routine is based on the “honesty” of the creator: MySQL does not check that a routine declared `DETERMINISTIC` is free of statements that produce nondeterministic results. However, misdeclaring a routine might affect results or affect performance. Declaring a nondeterministic routine as `DETERMINISTIC` might lead to unexpected results by causing the optimizer to make incorrect execution plan choices. Declaring a deterministic routine as `NONDETERMINISTIC` might diminish performance by causing available optimizations not to be used.

If binary logging is enabled, the `DETERMINISTIC` characteristic affects which routine definitions MySQL accepts. See Section 27.7, “Stored Program Binary Logging”.

A routine that contains the `NOW()` function (or its synonyms) or `RAND()` is nondeterministic, but it might still be replication-safe. For `NOW()`, the binary log includes the timestamp and replicates correctly. `RAND()` also replicates correctly as long as it is called only a single time during the execution of a routine. (You can consider the routine execution timestamp and random number seed as implicit inputs that are identical on the source and replica.)

Several characteristics provide information about the nature of data use by the routine. In MySQL, these characteristics are advisory only. The server does not use them to constrain what kinds of statements a routine is permitted to execute.

- **CONTAINS SQL** indicates that the routine does not contain statements that read or write data. This is the default if none of these characteristics is given explicitly. Examples of such statements are `SET @x = 1` or `DO RELEASE_LOCK( 'abc' )`, which execute but neither read nor write data.
- **NO SQL** indicates that the routine contains no SQL statements.
- **READS SQL DATA** indicates that the routine contains statements that read data (for example, SELECT), but not statements that write data.
- **MODIFIES SQL DATA** indicates that the routine contains statements that may write data (for example, INSERT or DELETE).

The **SQL SECURITY** characteristic can be **DEFINER** or **INVOKER** to specify the security context; that is, whether the routine executes using the privileges of the account named in the routine **DEFINER** clause or the user who invokes it. This account must have permission to access the database with which the routine is associated. The default value is **DEFINER**. The user who invokes the routine must have the EXECUTE privilege for it, as must the **DEFINER** account if the routine executes in definer security context.

The **DEFINER** clause specifies the MySQL account to be used when checking access privileges at routine execution time for routines that have the **SQL SECURITY DEFINER** characteristic.

If the **DEFINER** clause is present, the **user** value should be a MySQL account specified as `'user_name'@'host_name'`, CURRENT_USER, or CURRENT_USER(). The permitted **user** values depend on the privileges you hold, as discussed in Section 27.6, “Stored Object Access Control”. Also see that section for additional information about stored routine security.

If the **DEFINER** clause is omitted, the default definer is the user who executes the CREATE PROCEDURE or CREATE FUNCTION statement. This is the same as specifying **DEFINER = CURRENT_USER** explicitly.

Within the body of a stored routine that is defined with the **SQL SECURITY DEFINER** characteristic, the CURRENT_USER function returns the routine's **DEFINER** value. For information about user auditing within stored routines, see Section 8.2.23, “SQL-Based Account Activity Auditing”.

Consider the following procedure, which displays a count of the number of MySQL accounts listed in the `mysql.user` system table:

```
CREATE DEFINER = 'admin'@'localhost' PROCEDURE account_count()  
BEGIN  
    SELECT 'Number of accounts:', COUNT(*) FROM mysql.user;  
END;
```

The procedure is assigned a `DEFINER` account of 'admin'@'localhost' no matter which user defines it. It executes with the privileges of that account no matter which user invokes it (because the default security characteristic is `DEFINER`). The procedure succeeds or fails depending on whether invoker has the EXECUTE privilege for it and 'admin'@'localhost' has the SELECT privilege for the `mysql.user` table.

Now suppose that the procedure is defined with the `SQL SECURITY INVOKER` characteristic:

```
CREATE DEFINER = 'admin'@'localhost' PROCEDURE account_count()  
SQL SECURITY INVOKER  
BEGIN  
    SELECT 'Number of accounts:', COUNT(*) FROM mysql.user;  
END;
```

The procedure still has a `DEFINER` of 'admin'@'localhost', but in this case, it executes with the privileges of the invoking user. Thus, the procedure succeeds or fails depending on whether the invoker has the EXECUTE privilege for it and the SELECT privilege for the `mysql.user` table.

By default, when a routine with the `SQL SECURITY DEFINER` characteristic is executed, MySQL Server does not set any active roles for the MySQL account named in the `DEFINER` clause, only the default roles. The exception is if the activate_all_roles_on_login system variable is enabled, in which case MySQL Server sets all roles granted to the `DEFINER` user, including mandatory roles. Any privileges granted through roles are therefore not checked by default when the CREATE PROCEDURE or CREATE FUNCTION statement is issued. For stored programs, if execution should occur with roles different from the default, the program body can execute SET ROLE to activate the required roles. This must be done with caution since the privileges assigned to roles can be changed.

The server handles the data type of a routine parameter, local routine variable created with DECLARE, or function return value as follows:

- Assignments are checked for data type mismatches and overflow. Conversion and overflow problems result in warnings, or errors in strict SQL mode.
- Only scalar values can be assigned. For example, a statement such as `SET x = (SELECT 1, 2)` is invalid.

- For character data types, if `CHARACTER SET` is included in the declaration, the specified character set and its default collation is used. If the `COLLATE` attribute is also present, that collation is used rather than the default collation.

If `CHARACTER SET` and `COLLATE` are not present, the database character set and collation in effect at routine creation time are used. To avoid having the server use the database character set and collation, provide an explicit `CHARACTER SET` and a `COLLATE` attribute for character data parameters.

If you alter the database default character set or collation, stored routines that are to use the new database defaults must be dropped and recreated.

The database character set and collation are given by the value of the `character_set_database` and `collation_database` system variables. For more information, see Section 12.3.3, “Database Character Set and Collation”.